Attention Is All You Need（注意力是你所需要的一切）

作者：Ashish Vaswani、Noam Shazeer、Niki Parmar、Jakob Uszkoreit、Llion Jones、Aidan N. Gomez、Łukasz Kaiser、Illia Polosukhin

单位：Google Brain、Google Research、University of Toronto

Abstract（摘要）

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

主导的序列转导模型是基于复杂的递归或卷积神经网络，包含编码器和解码器。性能最佳的模型还通过注意力机制连接编码器和解码器。我们提出一种新的简单网络架构——Transformer，它完全基于注意力机制，摒弃了递归和卷积。在两项机器翻译任务上的实验表明，该模型质量更优，且更易于并行化，训练时间大幅缩短。我们的模型在WMT 2014英德翻译任务中达到28.4 BLEU，比包括集成模型在内的现有最佳结果提升超过2 BLEU；在WMT 2014英法翻译任务中，经8块GPU训练3.5天后，单模型BLEU得分达41.0，创下新的最先进水平，而训练成本仅为文献中最佳模型的一小部分。

1 Introduction（介绍）

Recurrent neural networks, long short-term memory [12] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [29, 2, 5]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [31, 21, 13].

循环神经网络，特别是长短时记忆（LSTM）[12]和门控递归神经网络（GRU）[7]，已被确立为序列建模和转导问题（如语言建模和机器翻译）的最先进方法[29, 2, 5]。此后，众多研究持续推动递归语言模型和编码器-解码器架构的发展边界[31, 21, 13]。

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t, as a function of the previous hidden state h_{t-1} and the input for position t. This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks [18] and conditional computation [26], while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains.

递归模型通常沿输入和输出序列的符号位置分解计算。将位置与计算时间步对齐，模型生成一系列隐藏状态 h_t ，其值依赖于前一隐藏状态 h_{t-1} 和位置t的输入。这种固有的顺序性导致训练样本内部无法并行化，在序列较长时这一问题尤为关键——内存限制会制约跨样本的批处理效率。近期研究通过因式分解技巧[18]和条件计算[26]大幅提升了计算效率，后者还同时优化了模型性能，但顺序计算的根本约束仍未解决。

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences [2, 16]. In all but a few cases [22], however, such attention mechanisms are used in conjunction with a recurrent network.

注意力机制已成为各类任务中高性能序列建模和转导模型的核心组件，能够建模输入或输出序列中任意距离的依赖关系[2, 16]。但除少数情况外[22]，这类注意力机制均需与递归网络配合使用。

In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs.

本文提出Transformer架构，它摒弃递归，完全依赖注意力机制捕获输入与输出之间的全局依赖。该架构支持更高程度的并行化，仅需8块P100 GPU训练12小时，就能在翻译质量上达到新的最先进水平。

2 Background（背景）

The goal of reducing sequential computation also forms the foundation of the Extended Neural GPU [20], ByteNet [15] and ConvS2S [8], all of which use convolutional neural networks as basic building block, computing hidden representations in parallel for all input and output positions. In these models, the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes it more difficult to learn dependencies between distant positions [11]. In the Transformer this is reduced to a constant number of operations, albeit at the cost of reduced effective resolution due to averaging attention-weighted positions, an effect we counteract with Multi-Head Attention as described in section 3.2.

减少顺序计算的目标也是Extended Neural GPU[20]、ByteNet[15]和ConvS2S[8]的核心设计理念，这些模型均以卷积神经网络为基础组件，对所有输入和输出位置并行计算隐藏表征。在这些模型中，关联任意两个输入或输出位置信号所需的操作数随位置距离增长——ConvS2S呈线性增长，ByteNet呈对数增长，这使得学习远距离依赖关系更加困难[11]。而Transformer将这一操作数降至常数级别，尽管注意力加权平均会导致有效分辨率降低，但我们通过3.2节所述的多头注意力机制抵消了这一影响。

Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-

attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations [4, 22, 23, 19].

自注意力（又称内部注意力）是一种关联单个序列不同位置以计算序列表征的注意力机制，已成功应用于阅读理解、抽象摘要、文本蕴含和任务无关句子表征学习等多个任务[4, 22, 23, 19]。

End-to-end memory networks are based on a recurrent attention mechanism instead of sequence-aligned recurrence and have been shown to perform well on simple-language question answering and language modeling tasks [28].

端到端记忆网络基于递归注意力机制而非序列对齐递归，在简单语言问答和语言建模任务中表现优异[28]。

To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence-aligned RNNs or convolution. In the following sections, we will describe the Transformer, motivate self-attention and discuss its advantages over models such as [14, 15] and [8].

然而据我们所知，Transformer是首个完全依赖自注意力计算输入和输出表征的转导模型，无需使用序列对齐的RNN或卷积。后续章节将详细描述Transformer，阐述自注意力的设计动机，并讨论其相对于[14, 15, 8]等模型的优势。

3 Model Architecture（模型架构）

Most competitive neural sequence transduction models have an encoder-decoder structure [5, 2, 29]. Here, the encoder maps an input sequence of symbol representations $(x_1, ..., x_n)$ to a sequence of continuous representations $z = (z_1, ..., z_n)$. Given z, the decoder then generates an output sequence $(y_1, ..., y_m)$ of symbols one element at a time. At each step the model is auto-regressive [9], consuming the previously generated symbols as additional input when generating the next.

大多数高性能神经序列转导模型均采用编码器-解码器结构[5, 2, 29]。编码器将输入符号表征序列 $(x_1, ..., x_n)$ 映射为连续表征序列 $z = (z_1, ..., z_n)$；解码器基于z逐元素生成输出符号序列 $(y_1, ..., y_m)$。模型在每一步均具有自回归特性[9]，生成下一个元素时会将已生成的符号作为额外输入。

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

Transformer沿用这一整体架构，编码器和解码器均由堆叠的自注意力层和逐点全连接层构成，分别如图1的左半部分和右半部分所示。

3.1 Encoder and Decoder Stacks（编码器和解码器堆栈）

Encoder（编码器）

The encoder is composed of a stack of $N = 6$ identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection [10] around each of the two sub-layers, followed by layer normalization [1]. That is, the output of each sub-layer is $\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$, where $\mathrm{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\text{model}} = 512$.

编码器由 $N = 6$ 个相同层堆叠而成，每层包含两个子层：第一层是多头自注意力机制，第二层是逐点全连接前馈网络。每个子层周围均添加残差连接[10]，随后进行层归一化[1]。即每个子层的输出为 $\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$ ，其中 $\mathrm{Sublayer}(x)$ 为子层自身实现的函数。为适配残差连接，模型中所有子层及嵌入层的输出维度均设为 $d_{\text{model}} = 512$ 。

Decoder（解码器）

The decoder is also composed of a stack of $N = 6$ identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i.

解码器同样由 $N = 6$ 个相同层堆叠而成。除编码器层的两个子层外，解码器新增第三个子层——对编码器堆栈的输出执行多头注意力。与编码器类似，每个子层周围均添加残差连接和层归一化。此外，解码器的自注意力子层经过修改，防止位置关注后续位置。这种掩码机制结合输出嵌入偏移一个位置的设计，确保位置i的预测仅依赖于小于i的已知输出位置。

3.2 Attention（注意力）

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

注意力函数可描述为将查询（query）和一组键值对（key-value）映射为输出，其中查询、键、值和输出均为向量。输出是值的加权和，每个值的权重由查询与对应键的兼容性函数计算得出。

3.2.1 Scaled Dot-Product Attention（缩放点积注意力）

We call our particular attention "Scaled Dot-Product Attention" (Figure 2). The input consists of queries and keys of dimension d_k, and values of dimension d_v. We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

我们将所提出的注意力机制称为“缩放点积注意力”（图2左）。输入包含维度为 d_k 的查询和键，以及维度为 d_v 的值。计算查询与所有键的点积，除以 $\sqrt{d_k}$ 后通过Softmax函数得到值的权重。

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q. The keys and values are also packed together into matrices K and V. We compute the matrix of outputs as:

[

\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) V \tag{1}

]

实际应用中，我们将一组查询打包为矩阵Q，键和值分别打包为矩阵K和V，批量计算注意力输出：

[

\text{注意力}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) V \tag{1}

]

The two most commonly used attention functions are additive attention [2], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

两种最常用的注意力函数是加法注意力[2]和点积（乘法）注意力。点积注意力与本文算法的唯一区别是缺少缩放因子 $\frac{1}{\sqrt{d_k}}$。加法注意力通过含单个隐藏层的前馈网络计算兼容性函数，两者理论复杂度相近，但点积注意力可借助高度优化的矩阵乘法实现，实际速度更快、空间效率更高。

While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k [3]. We suspect that for large values of d_k, the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients. To counteract this effect, we scale the dot products by $\frac{1}{\sqrt{d_k}}$.

当 d_k 较小时，两种机制性能相近；但 d_k 较大时，未缩放的点积注意力性能劣于加法注意力[3]。我们推测，d_k 较大时，点积结果的数值会急剧增大，导致Softmax函数进入梯度极小的区域。为抵消这一影响，我们用 $\frac{1}{\sqrt{d_k}}$ 对点积结果进行缩放。

3.2.2 Multi-Head Attention（多头注意力）

Instead of performing a single attention function with d_{model}-dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k, d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v-dimensional output values. These are concatenated and once again projected, resulting in the final values, as depicted in Figure 2.

我们未使用 d_{model} 维的键、值和查询执行单一注意力函数，而是通过h组不同的可学习线性投影，将查询、键和值分别投影到 d_k 、 d_k 和 d_v 维，再对每组投影后的查询、键和值并行执行注意力函数，得到h个 d_v 维输出。将这些输出拼接后再次投影，得到最终结果（图2右）。

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

多头注意力使模型能同时关注不同位置、不同表征子空间的信息，而单头注意力的平均效应会限制这一能力。

[

\text{MultiHead}(Q, K, V) = \text{Concat}\left(\text{head}_1, ..., \text{head}_h\right) W^O

]

[

\text{where head}_i = \text{Attention}\left(Q W_i^Q, K W_i^K, V W_i^V\right)

]

其中，投影参数矩阵为 $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 、 $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 、 $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ 和 $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ 。

In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_{\text{model}}/h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

本文中，我们设置 $h = 8$ 个并行注意力层（即8个头），每个头的 $d_k = d_v = d_{\text{model}}/h = 64$ 。由于每个头的维度降低，总计算成本与全维度单头注意力相近。

3.2.3 Applications of Attention in our Model（注意力在模型中的应用）

The Transformer uses multi-head attention in three different ways:

- In "encoder-decoder attention" layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models such as [31, 2, 8].
- The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.

- Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections. See Figure 2.

Transformer通过三种方式使用多头注意力：

- 编码器-解码器注意力层：查询来自解码器前一层，键和值来自编码器输出，使解码器每个位置能关注输入序列的所有位置，模仿[31, 2, 8]等序列到序列模型的典型编码器-解码器注意力机制。
- 编码器自注意力层：键、值和查询均来自编码器前一层输出，使编码器每个位置能关注前一层的所有位置。
- 解码器自注意力层：键、值和查询均来自解码器前一层输出，但仅允许关注当前位置及之前的位置。为保持自回归特性，我们在缩放点积注意力中通过掩码（将非法连接对应的Softmax输入设为$-\infty$）阻止信息向左流动（见图2）。

3.3 Position-wise Feed-Forward Networks（逐点前馈网络）

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

[

\text{FFN}(x) = \max(0, x W_1 + b_1) W_2 + b_2 \tag{2}

]

除注意力子层外，编码器和解码器的每层均包含一个全连接前馈网络，该网络对每个位置独立执行相同操作，由两个线性变换和中间的ReLU激活组成：

[

\text{FFN}(x) = \max(0, x W_1 + b_1) W_2 + b_2 \tag{2}

]

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is $d_{\text{model}} = 512$, and the inner-layer has dimensionality $d_{ff} = 2048$.

不同位置共享相同的线性变换，但不同层使用不同参数。这一结构也可描述为两个核大小为1的卷积。输入和输出维度为 $d_{\text{model}} = 512$，内层维度为 $d_{ff} = 2048$。

3.4 Embeddings and Softmax（嵌入和Softmax）

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_{model}. We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation, similar to [24]. In the embedding layers, we multiply those weights by $\sqrt{d_{\text{model}}}$.

与其他序列转导模型类似，我们通过可学习嵌入将输入和输出符号转换为 d_{model} 维向量，再通过可学习线性变换和Softmax函数将解码器输出转换为下一个符号的预测概率。借鉴[24]，我们让输入嵌入层、输出嵌入层与Softmax前的线性变换共享权重矩阵，并在嵌入层中将权重乘以 $\sqrt{d_{\text{model}}}$。

3.5 Positional Encoding（位置编码）

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add "positional encodings" to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed [8].

由于模型不含递归和卷积，为使模型利用序列顺序信息，我们需向编码器和解码器堆栈底部的输入嵌入添加“位置编码”。位置编码与嵌入维度相同（$d_{\text{model}} = 512$），因此可直接相加。位置编码的选择多样，包括可学习型和固定型[8]。

In this work, we use sine and cosine functions of different frequencies:

[

PE_{(pos, 2i)} = \sin\left(pos / 10000^{2i / d_{\text{model}}}\right)

]

[

PE_{(pos, 2i+1)} = \cos\left(pos / 10000^{2i / d_{\text{model}}}\right)

]

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k, PE_{pos+k} can be represented as a linear function of PE_{pos}.

本文采用不同频率的正弦和余弦函数作为位置编码：

[

PE_{(pos, 2i)} = \sin\left(pos / 10000^{2i / d_{\text{model}}}\right)

]

[

PE_{(pos, 2i+1)} = \cos\left(pos / 10000^{2i / d_{\text{model}}}\right)

]

其中pos为位置，i为维度。位置编码的每个维度对应一个正弦波，波长从 2π 到 $10000 \cdot 2\pi$ 呈几何级数分布。选择该函数的原因是，对于任意固定偏移k， PE_{pos+k} 可表示为 PE_{pos} 的线性函数，便于模型学习相对位置依赖。

We also experimented with using learned positional embeddings [8] instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

我们也尝试了可学习位置编码[8]，发现两种编码方式结果几乎一致（见表3行（E））。最终选择正弦余弦编码，因其可使模型外推到训练过程中未见过的更长序列。

4 Why Self-Attention（为什么选择自注意力）

In this section we compare various aspects of self-attention layers to the recurrent and convolutional layers commonly used for mapping one variable-length sequence of symbol representations $(x_1, ..., x_n)$ to another sequence of equal length $(z_1, ..., z_n)$, with $x_i, z_i \in \mathbb{R}^d$, such as a hidden layer in a typical sequence transduction encoder or decoder. Motivating our use of self-attention we consider three desiderata.

本节将自注意力层与递归层、卷积层进行多方面比较——三者均用于将变长符号表征序列 $(x_1, ..., x_n)$ 映射为等长序列 $(z_1, ..., z_n)$（$x_i, z_i \in \mathbb{R}^d$），例如典型序列转导编码器或解码器的隐藏层。我们基于三个核心需求选择自注意力：

One is the total computational complexity per layer. Another is the amount of computation that can be parallelized, as measured by the minimum number of sequential operations required.

一是每层的总计算复杂度；二是可并行化的计算量（以所需的最小顺序操作数衡量）。

The third is the path length between long-range dependencies in the network. Learning long-range dependencies is a key challenge in many sequence transduction tasks. One key factor affecting the ability to learn such dependencies is the length of the paths forward and backward signals have to traverse in the network. The shorter these paths between any combination of positions in the input and output sequences, the easier it is to learn long-range dependencies [11]. Hence we also compare the maximum path length between any two input and output positions in networks composed of the different layer types.

三是网络中长距离依赖的路径长度。学习长距离依赖是许多序列转导任务的核心挑战，前向和后向信号在网络中的传播路径长度是关键影响因素——输入和输出序列任意位置组合的路径越短，越容易学习长距离依赖[11]。因此，我们还比较了不同层类型组成的网络中，任意两个输入和输出位置之间的最大路径长度。

Layer Type（层类型）	Complexity per Layer（每层复杂度）	Sequential Operations（顺序操作数）	Maximum Path L 大路径长度）
Self-Attention（自注意力）	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent（递归）	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional（卷积）	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)（受限自注意力）	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

As noted in Table 1, a self-attention layer connects all positions with a constant number of sequentially executed operations, whereas a recurrent layer requires $O(n)$ sequential operations. In terms of computational complexity, self-attention layers are faster than recurrent layers when the sequence length n is smaller than the representation dimensionality d, which is most often the case with sentence representations used by state-of-the-art models in machine translations, such as word-piece [31] and byte-pair [25] representations. To improve

computational performance for tasks involving very long sequences, self-attention could be restricted to considering only a neighborhood of size r in the input sequence centered around the respective output position. This would increase the maximum path length to $O(n/r)$. We plan to investigate this approach further in future work.

如表1所示，自注意力层以恒定数量的顺序操作连接所有位置，而递归层需 $O(n)$ 个顺序操作。计算复杂度方面，当序列长度n小于表征维度d时（机器翻译中最先进模型的句子表征如词片[31]、字节对[25]均满足这一条件），自注意力层比递归层更快。对于超长序列任务，可将自注意力限制为仅关注输入序列中以输出位置为中心、大小为r的邻域，此时最大路径长度增至 $O(n/r)$，我们计划在未来工作中进一步研究这一方法。

A single convolutional layer with kernel width $k < n$ does not connect all pairs of input and output positions. Doing so requires a stack of $O(n/k)$ convolutional layers in the case of contiguous kernels, or $O(\log_k(n))$ in the case of dilated convolutions [15], increasing the length of the longest paths between any two positions in the network. Convolutional layers are generally more expensive than recurrent layers, by a factor of k. Separable convolutions [6], however, decrease the complexity considerably, to $O(k \cdot n \cdot d + n \cdot d^2)$. Even with $k = n$, however, the complexity of a separable convolution is equal to the combination of a self-attention layer and a point-wise feed-forward layer, the approach we take in our model.

核宽度 $k < n$ 的单一卷积层无法连接所有输入输出位置对：连续核需堆叠 $O(n/k)$ 个卷积层，扩张卷积[15]需堆叠 $O(\log_k(n))$ 个卷积层，这会增加网络中任意两个位置的最长路径长度。卷积层的计算成本通常是递归层的k倍，但可分离卷积[6]能将复杂度降至 $O(k \cdot n \cdot d + n \cdot d^2)$。即便 $k = n$，可分离卷积的复杂度仍与本文模型采用的“自注意力层+逐点前馈层”组合相当。

As side benefit, self-attention could yield more interpretable models. We inspect attention distributions from our models and present and discuss examples in the appendix. Not only do individual attention heads clearly learn to perform different tasks, many appear to exhibit behavior related to the syntactic and semantic structure of the sentences.

此外，自注意力还能提升模型的可解释性。我们分析了模型的注意力分布（附录中提供示例及讨论），发现单个注意力头不仅能明确学习执行不同任务，许多头还表现出与句子句法和语义结构相关的行为。

5 Training（训练）

This section describes the training regime for our models.

本节详细说明模型的训练方案。

5.1 Training Data and Batching（训练数据和批处理）

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. Sentences were encoded using byte-pair encoding [3], which has a shared source-target vocabulary of about 37000 tokens. For English-French, we used the significantly larger WMT 2014 English-French dataset consisting of 36M sentences and split tokens into a 32000 word-piece vocabulary [31]. Sentence pairs were batched together by approximate sequence length. Each training batch contained a set of sentence pairs containing approximately 25000 source tokens and 25000 target tokens.

训练数据采用标准WMT 2014数据集：英德语料含约450万个句子对，使用字节对编码[3]，源-目标共享词汇量约37000个符号；英法语料含3600万个句子对，使用词片编码[31]，词汇量32000个。按序列长度近似分组批处理，每个训练批次包含约25000个源符号和25000个目标符号。

5.2 Hardware and Schedule（硬件和训练进度）

We trained our models on one machine with 8 NVIDIA P100 GPUs. For our base models using the hyperparameters described throughout the paper, each training step took about 0.4 seconds. We trained the base models for a total of 100,000 steps or 12 hours. For our big models,(described on the bottom line of table 3), step time was 1.0 seconds. The big models were trained for 300,000 steps (3.5 days).

训练硬件为单台含8块NVIDIA P100 GPU的机器：基础模型（本文所述超参数）每步训练约0.4秒，共训练10万步（12小时）；大型模型（表3最后一行）每步训练1.0秒，共训练30万步（3.5天）。

5.3 Optimizer（优化器）

We used the Adam optimizer [17] with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$. We varied the learning rate over the course of training, according to the formula:

[

\text{lrate} = d_{\text{model}}^{-0.5} \cdot \min(\text{step_num}^{-0.5}, \text{step_num} \cdot \text{warmup_steps}^{-1.5}) \tag{3}

]

采用Adam优化器[17]，参数设置为 $\beta_1 = 0.9$、$\beta_2 = 0.98$、$\epsilon = 10^{-9}$。学习率随训练过程动态调整，公式如下：

[

\text{学习率} = d_{\text{model}}^{-0.5} \cdot \min(\text{步数}^{-0.5}, \text{步数} \cdot \text{预热步数}^{-1.5}) \tag{3}

]

This corresponds to increasing the learning rate linearly for the first warmup_steps training steps, and decreasing it thereafter proportionally to the inverse square root of the step number. We used warmup_steps = 4000.

该公式表示：前4000步（预热步数）学习率线性增长，之后随步数的平方根倒数成比例下降。

5.4 Regularization（正则化）

We employ three types of regularization during training:

- Residual Dropout: We apply dropout [27] to the output of each sub-layer, before it is added to the sub-layer input and normalized. In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of $P_{\text{drop}} = 0.1$.
- Label Smoothing: During training, we employed label smoothing of value $\epsilon_{ls} = 0.1$ [30]. This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score.

训练过程采用两种正则化策略：

- 残差丢弃（Residual Dropout）：对每个子层输出、编码器和解码器的嵌入与位置编码之和执行丢弃操作[27]，基础模型的丢弃率 $P_{\text{drop}} = 0.1$。
- 标签平滑（Label Smoothing）：标签平滑系数 $\epsilon_{ls} = 0.1$ [30]，这会提高困惑度（模型不确定性增加），但能提升准确率和BLEU得分。

6 Results（实验结果）

6.1 Machine Translation（机器翻译结果）

On the WMT 2014 English-to-German translation task, the big transformer model (Transformer (big) in Table 2) outperforms the best previously reported models (including ensembles) by more than 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. The configuration of this model is listed in the bottom line of Table 3. Training took 3.5 days on 8 P100 GPUs. Even our base model surpasses all previously published models and ensembles, at a fraction of the training cost of any of the competitive models.

在WMT 2014英德翻译任务中，大型Transformer模型（表2中“Transformer (big)”）的BLEU得分为28.4，比此前所有已报道模型（包括集成模型）提升超过2.0，创下新纪录。该模型配置见表3最后一行，经8块P100 GPU训练3.5天完成。即使是基础模型，也超越了所有此前发表的模型和集成模型，且训练成本仅为同类竞争模型的一小部分。

On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.0, outperforming all of the previously published single models, at less than 1/4 the training cost of the previous state-of-the-art model. The Transformer (big) model trained for English-to-French used dropout rate 0.3 instead of 0.1.

在WMT 2014英法翻译任务中，大型模型的BLEU得分为41.0，超越所有此前发表的单模型，训练成本不足此前最先进模型的1/4。该任务的大型Transformer模型将丢弃率调整为0.3（基础模型为0.1）。

For the base models, we used a single model obtained by averaging the last 5 checkpoints, which were written at 10-minute intervals. For the big models, we averaged the last 20 checkpoints. We used beam search with a beam size of 4 and length penalty $\alpha = 0.6$ [31]. These hyperparameters were chosen after experimentation on the development set. We set the maximum output length during inference to input length + 50, but terminate early when possible [31].

基础模型采用最后5个检查点（每10分钟保存一个）的平均结果；大型模型采用最后20个检查点的平均结果。推理阶段使用束搜索，束大小为4，长度惩罚系数 $\alpha = 0.6$ [31]，这些超参数经开发集实验确定。输出序列最大长度设为输入长度+50，支持早停机制[31]。

Model（模型）	BLEU		Training Cost (FL 成本）
	EN-DE（英德）	EN-FR（英法）	EN-DE（英德）
ByteNet [15]	23.75	-	-
Deep-Att + PosUnk [32]	-	39.2	-
GNMT + RL [31]	24.6	39.92	$2.3 \cdot 10^{19}$
ConvS2S [8]	25.16	40.46	$9.6 \cdot 10^{18}$
MoE [26]	26.03	40.56	$2.0 \cdot 10^{19}$
Deep-Att + PosUnk Ensemble [32]	-	40.4	-
GNMT + RL Ensemble [31]	26.30	41.16	$1.8 \cdot 10^{20}$
ConvS2S Ensemble [8]	26.36	41.29	$7.7 \cdot 10^{19}$

Transformer (base model)（基础模型）	27.3	38.1	$3.3 \cdot 10^{18}$
Transformer (big)（大型模型）	28.4	41.0	$2.3 \cdot 10^{19}$

Table 2 summarizes our results and compares our translation quality and training costs to other model architectures from the literature. We estimate the number of floating point operations used to train a model by multiplying the training time, the number of GPUs used, and an estimate of the sustained single-precision floating-point capacity of each GPU.

表2总结了实验结果，并与文献中的其他模型架构进行了翻译质量和训练成本对比。训练成本（浮点运算次数）通过训练时间、GPU数量和单块GPU的持续单精度浮点运算能力估算得出。

6.2 Model Variations（模型变体实验）

To evaluate the importance of different components of the Transformer, we varied our base model in different ways, measuring the change in performance on English-to-German translation on the development set, newstest2013. We used beam search as described in the previous section, but no checkpoint averaging. We present these results in Table 3.

为评估Transformer各组件的重要性，我们对基础模型进行了多种变体修改，在英德翻译开发集newstest2013上测试性能变化。推理阶段采用前述束搜索策略，但不进行检查点平均，结果见表3。

In Table 3 rows (A), we vary the number of attention heads and the attention key and value dimensions, keeping the amount of computation constant, as described in Section 3.2.2. While single-head attention is 0.9 BLEU worse than the best setting, quality also drops off with too many heads.

表3行（A）展示了注意力头数h与键/值维度的变化（保持计算量恒定，见3.2.2节）：单头注意力比最佳设置低0.9 BLEU，头数过多时性能也会下降。

In Table 3 rows (B), we observe that reducing the attention key size d_k hurts model quality. This suggests that determining compatibility is not easy and that a more sophisticated

compatibility function than dot product may be beneficial. We further observe in rows (C) and (D) that, as expected, bigger models are better, and dropout is very helpful in avoiding over-fitting. In row (E) we replace our sinusoidal positional encoding with learned positional embeddings [8], and observe nearly identical results to the base model.

表3行（B）显示，减小注意力键维度 d_k 会损害模型质量，说明兼容性计算并非易事，可能需要比点积更复杂的兼容性函数。行（C）和（D）验证了预期：模型规模越大性能越好，丢弃操作能有效避免过拟合。行（E）用可学习位置编码[8]替代正弦余弦编码，结果与基础模型几乎一致。

	N	d_{model}	dff
base（基础模型）	6	512	2048
(A)	6	512	2048
	6	512	2048
	6	512	2048
	6	512	2048
(B)	6	512	2048
	6	512	2048
(C)	2	512	2048
	4	512	2048
	8	512	2048
	6	256	1024
	6	1024	4096
	6	1024	2048
	6	4096	2048
(D)	6	512	2048
	6	512	2048
	6	512	2048
	6	512	2048

(E)	6	512	2048
big（大型模型）	6	1024	4096

7 Conclusion（结论）

In this work, we presented the Transformer, the first sequence transduction model based entirely on attention, replacing the recurrent layers most commonly used in encoder-decoder architectures with multi-headed self-attention.

本文提出Transformer，这是首个完全基于注意力的序列转导模型，用多头自注意力取代了编码器-解码器架构中最常用的递归层。

For translation tasks, the Transformer can be trained significantly faster than architectures based on recurrent or convolutional layers. On both WMT 2014 English-to-German and WMT 2014 English-to-French translation tasks, we achieve a new state of the art. In the former task our best model outperforms even all previously reported ensembles.

在翻译任务中，Transformer的训练速度显著快于基于递归或卷积层的架构。在WMT 2014英德和英法翻译任务中，我们均创下新的最先进水平，其中英德任务的最佳模型甚至超越了所有此前报道的集成模型。

We are excited about the future of attention-based models and plan to apply them to other tasks. We plan to extend the Transformer to problems involving input and output modalities other than text and to investigate local, restricted attention mechanisms to efficiently handle large inputs and outputs such as images, audio and video. Making generation less sequential is another research goals of ours.

我们对基于注意力的模型的未来充满期待，并计划将其应用于其他任务：扩展到文本以外的输入输出模态，研究局部受限注意力机制以高效处理图像、音频、视频等大型输入输出，以及降低生成过程的顺序性。

The code we used to train and evaluate our models is available at https://github.com/tensorflow/tensor2tensor.

训练和评估所用代码已开源：https://github.com/tensorflow/tensor2tensor.

Acknowledgements（致谢）

We are grateful to Nal Kalchbrenner and Stephan Gouws for their fruitful comments, corrections and inspiration.

感谢Nal Kalchbrenner和Stephan Gouws提供的有益建议、修正和启发。

References（参考文献）

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. arXiv preprint arXiv:1607.06450, 2016.

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. CoRR, abs/1409.0473, 2014.

[3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. CoRR, abs/1703.03906, 2017.

[4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. arXiv preprint arXiv:1601.06733, 2016.

[5] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. CoRR, abs/1406.1078, 2014.

[6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. arXiv preprint arXiv:1610.02357, 2016.

[7] Junyoung Chung, Çaglar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. CoRR, abs/1412.3555, 2014.

[8] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. arXiv preprint arXiv:1705.03122v2, 2017.

[9] Alex Graves. Generating sequences with recurrent neural networks. arXiv preprint arXiv:1308.0850, 2013.

[10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pages 770–778, 2016.

[11] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.

[12] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. Neural computation, 9(8):1735–1780, 1997.

[13] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. arXiv preprint arXiv:1602.02410, 2016.

[14] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In International Conference on Learning Representations (ICLR), 2016.

[15] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. arXiv preprint arXiv:1610.10099v2, 2017.

[16] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In International Conference on Learning Representations, 2017.

[17] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In ICLR, 2015.

[18] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. arXiv preprint arXiv:1703.10722, 2017.

[19] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. arXiv preprint arXiv:1703.03130, 2017.

[20] Samy Bengio, Łukasz Kaiser. Can active memory replace attention? In Advances in Neural Information Processing Systems (NIPS), 2016.

[21] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. arXiv preprint arXiv:1508.04025, 2015.

[22] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In Empirical Methods in Natural Language Processing, 2016.

[23] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. arXiv preprint arXiv:1705.04304, 2017.

[24] Ofir Press and Lior Wolf. Using the output embedding to improve language models. arXiv preprint arXiv:1608.05859, 2016.

[25] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. arXiv preprint arXiv:1508.07909, 2015.

[26] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. arXiv preprint arXiv:1701.06538, 2017.

[27] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15(1):1929–1958, 2014.

[28] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In Advances in Neural Information Processing Systems 28, pages 2440–2448, 2015.

[29] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In Advances in Neural Information Processing Systems, pages 3104–3112, 2014.

[30] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. CoRR, abs/1512.00567, 2015.

[31] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’ s neural machine translation system: Bridging the gap between human and machine translation. arXiv preprint arXiv:1609.08144, 2016.

[32] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. CoRR, abs/1606.04199, 2016.